

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

Table of Contents

Introduction **Page 1 to Page 2**

Task 1 – Merge Sort Diagram **Page 3**

Task 1 – Quick Sort Diagram **Page 4**

Task 1 – Insertion Sort Diagram **Page 5**

Task 2 - Dijkstra's Algorithm **Page 6 to Page 10**

Adjacency List **Page 11**

Edge Weight Table **Page 11**

Step by step for Dijkstra's Algorithm in table format **Page 12**

Dijkstra's Algorithm Results: Shortest Paths and Distances **Page 12**

Task 3:

(a) Create a Hash Table **Page 13**

(b) Create a Binary Search Tree (BST) **Page 14 to Page 15**

Conclusion **Page 15**

Reference List **Page 16 to 17**

Introduction

The following report is for the practical application of important algorithms and data structures to solve certain computational problems. This report covers efficient techniques for data processing and solving challenges, which are very common in computer science, divided into three major tasks.

The first task to be pursued is the implementation of Dijkstra's algorithm, which is one of the most effective ways to determine the shortest paths between nodes in a weighted

Name: Mohamed Ibrahim
Algorithms Report – Coursework 2

Assignment Title:

Date of last update: 30/12/2024

graph. The algorithm will be indispensable in transportation, navigation, and network

routing, among other fields dealing with path optimization problems. In this problem, a step-by-step breakdown will be pursued by calculation of the shortest distances and then visualization for clarity [1].

The second assignment deals with the hash table, which is a robust data structure for the fastest storage and retrieval. Construction of the hash table for letters in the name "Mohamed Ibrahim" shows how the collision could be effectively managed by the chaining method so that the integrity of the data is maintained even when several letters may be allocated to the same position.

The last exercise introduces the concept of hierarchical organization using a binary search tree. Create a BST from the letters in the name "Mohamed Ibrahim" to store them in a logical structure that would support fast searching and sorting. This activity will show how a BST can be used to organize data logically, based on left and right child placement rules for smaller and greater values, respectively.

Each problem is explained in detail, with pictures and emphasis on why these algorithms and data structures are developed. The report points out not only the technical steps involved but also their applicability to solve real-life problems, thus making the concepts more friendly and applicable [3].

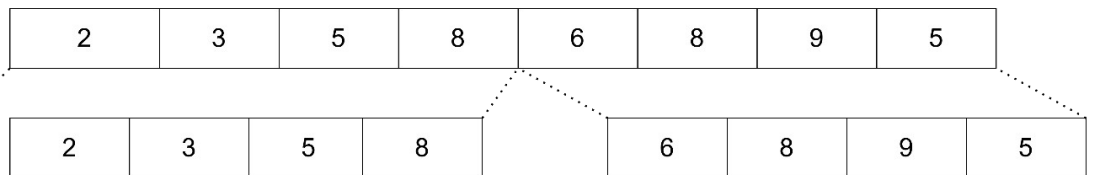
Date of last update: 30/12/2024

Task 1:

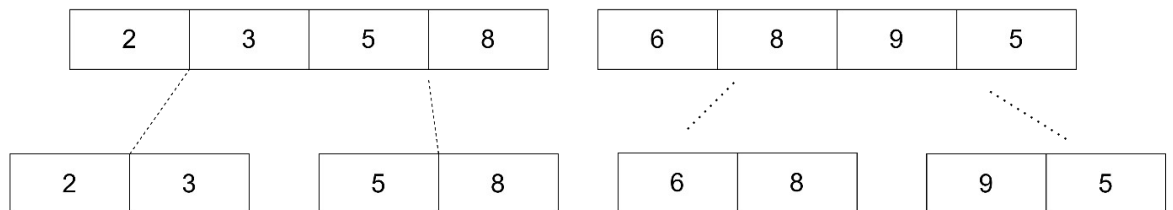
Merge sort

Step 1:

Split the array into two equal halves



Step 2: Split the sub arrays into four equal halves



Step 3: Merge the numbers into sorted sub arrays



Step 4: Merge the sorted sub arrays into the final sorted array



[2]

Quick Sort

Step 1: Make the first list of the array

2	3	5	8	6	8	9	5
---	---	---	---	---	---	---	---

Pivot

Step 2: Change the list so that it is changed based on the pivot which is 5

2	3	5	5	6	8	8	9
---	---	---	---	---	---	---	---

Step 3: Progress to quicksort to the sublist arrays

2	3
---	---

Pivot

6	8	8	9
---	---	---	---

Pivot

Final sorted list

2	3	5	5	6	8	8	9
---	---	---	---	---	---	---	---

Step 4: Elements are in the correct order based on the pivot, combine the sorted sub arrays

[4]

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

Name: Mohamed Ibrahim
Algorithms Report – Coursework 2

Assignment Title:

Date of last update: 30/12/2024

Insertion sort

Step 1:

The first element which is 2 is sorted already so you don't have to change it

2	3	5	8	6	8	9	5
---	---	---	---	---	---	---	---

Step 2:

Next, we are looking at the left of the array as there was no other elements in front of 2 which are less so the next element is 3 and because 3 is larger than 2, it stays in the same position.

2	3	5	8	6	8	9	5
---	---	---	---	---	---	---	---

Step 3:

5 is bigger than 2 and the element 3 so no positions needed change..

2	3	5	8	6	8	9	5
---	---	---	---	---	---	---	---

Step 4:

Look at the previous elements which are 2, 3 and 5 and these are less than 8 so no changes are made

2	3	5	8	6	8	9	5
---	---	---	---	---	---	---	---

Step 5:

Look at the elements which are 2, 3, 5 and 8 and since 6 is smaller than 8 and bigger than 5 its placed between 5 and 8.

2	3	5	6	8	8	9	5
---	---	---	---	---	---	---	---

Step 6:

Look at the elements that are before 8 which are 2, 3, 5, 6, and 8 and 8 is equal to the previous element 8 so no elements are changed

2	3	5	6	8	8	9	5
---	---	---	---	---	---	---	---

Step 7:

Look at the previous elements which are before the element 9 and these are 2, 3, 5, 6, 8 and 8 and because 9 is bigger than all of the elements, no changes are made.

2	3	5	6	8	8	9	5
---	---	---	---	---	---	---	---

Step 8:

Look at the previous elements which are 2, 3, 5, 6, 8, 8 and 9, since 5 is smaller than 6 and is equal to the first 5, it is put after the first five. The new array becomes [2, 3, 5, 5, 6, 8, 8, 9].

2	3	5	5	6	8	8	9
---	---	---	---	---	---	---	---

The final sorted array is
[2, 3, 5, 5, 6, 8, 8, 9]

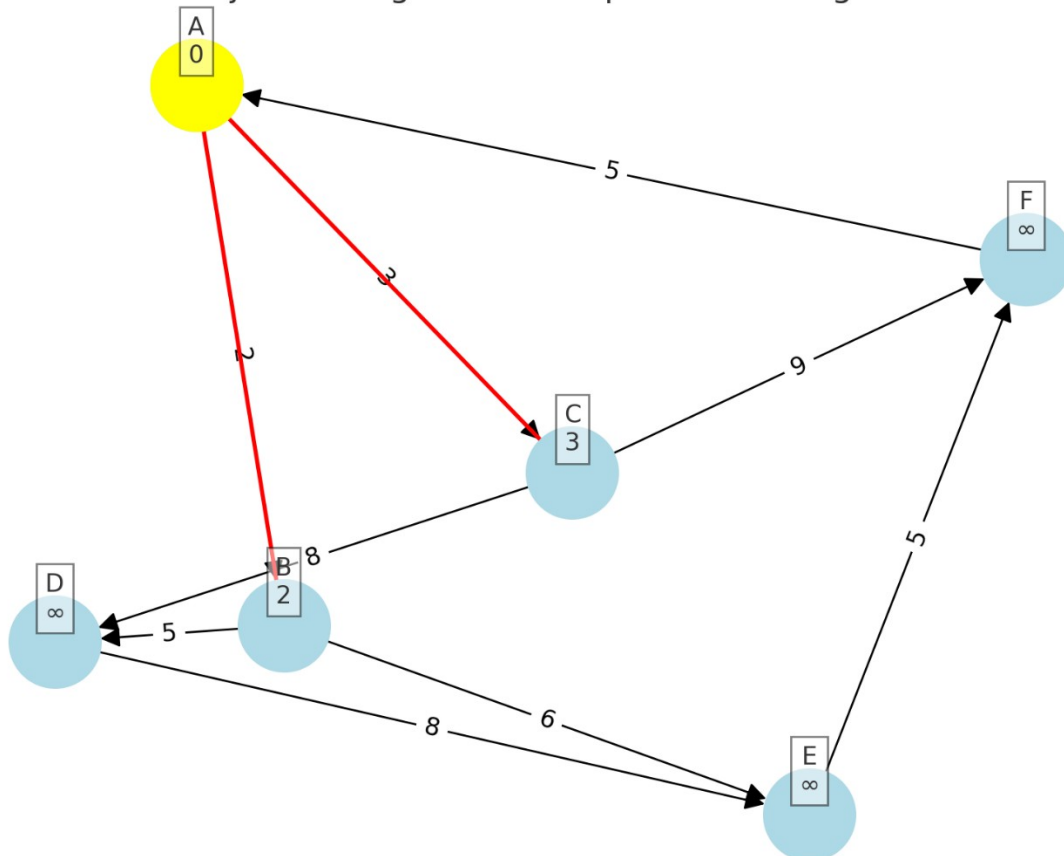
Date of last update: 30/12/2024

[5]

Task 2: Dijkstra's Algorithm.

Steps by step

Dijkstra's Algorithm - Step 1: Processing A

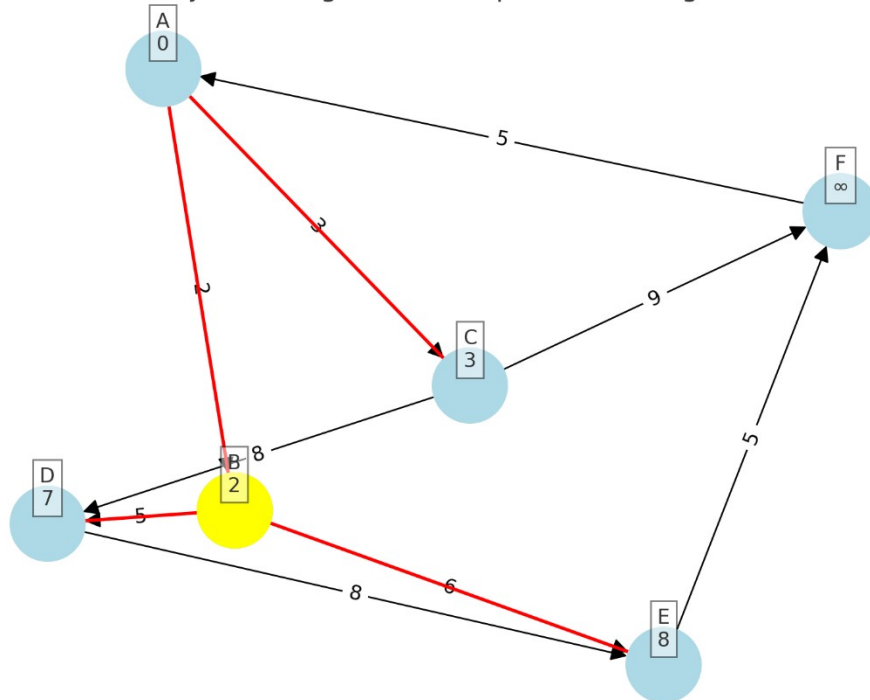


Priority Queue: [('B', 2), ('C', 3)]

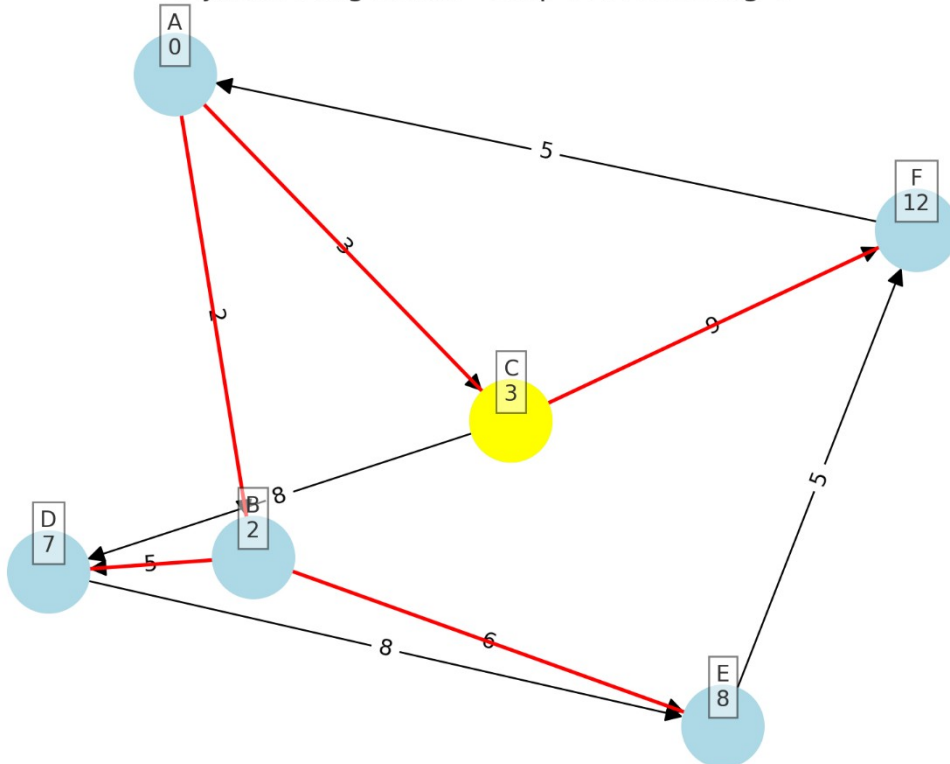
[6]

Date of last update: 30/12/2024

Dijkstra's Algorithm - Step 2: Processing B

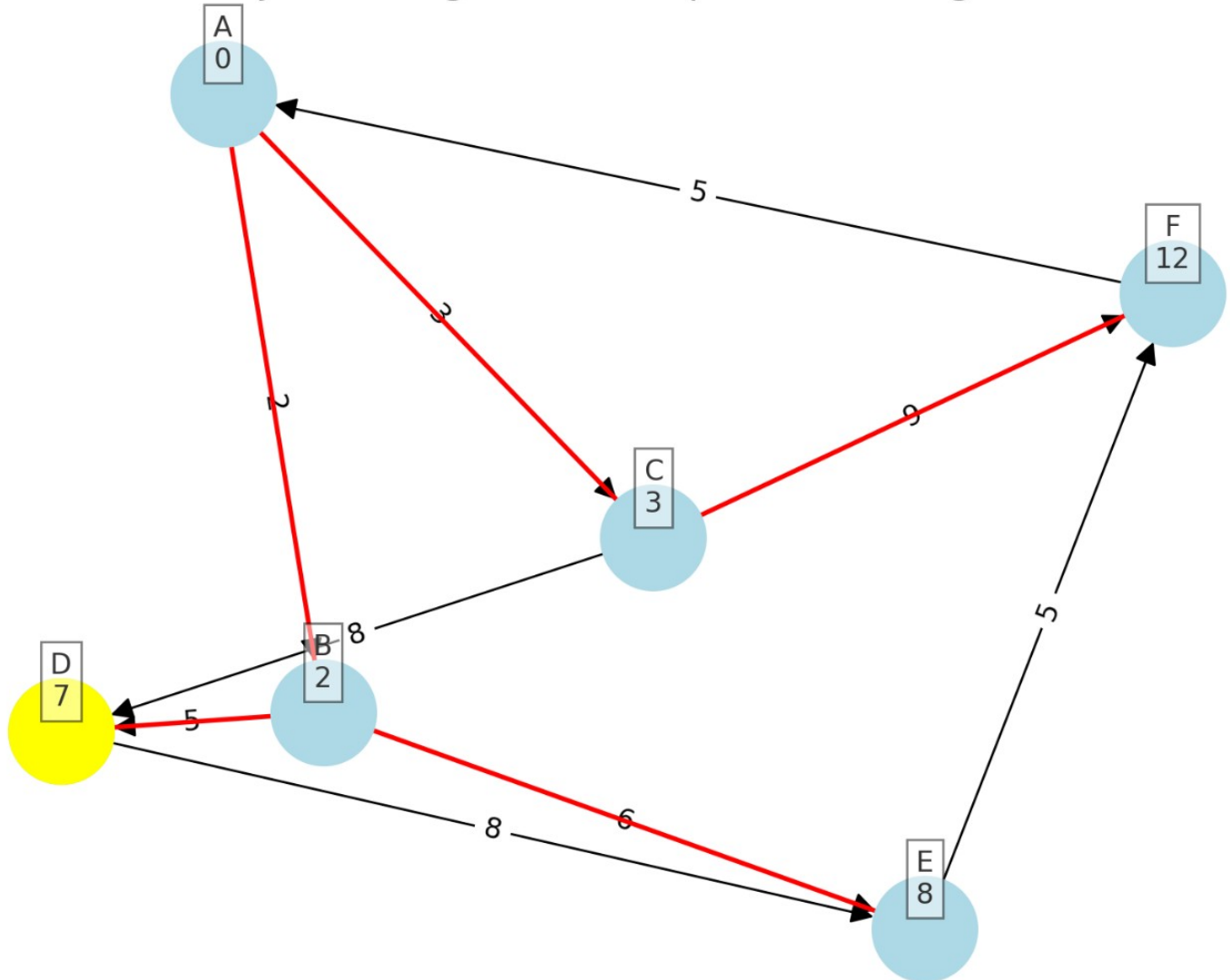


Dijkstra's Algorithm - Step 3: Processing C



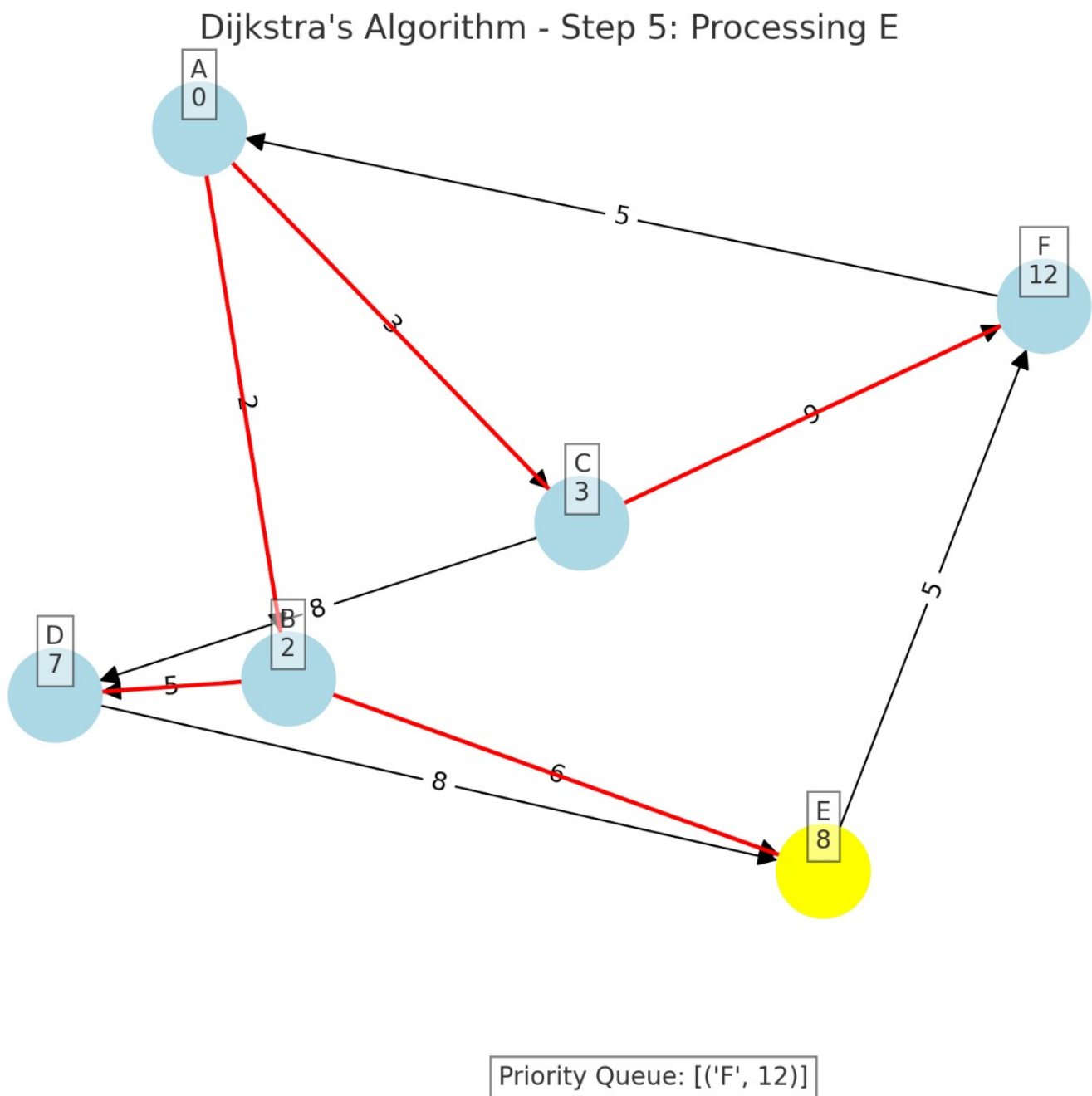
Priority Queue: [('D', 7), ('E', 8), ('F', 12)]

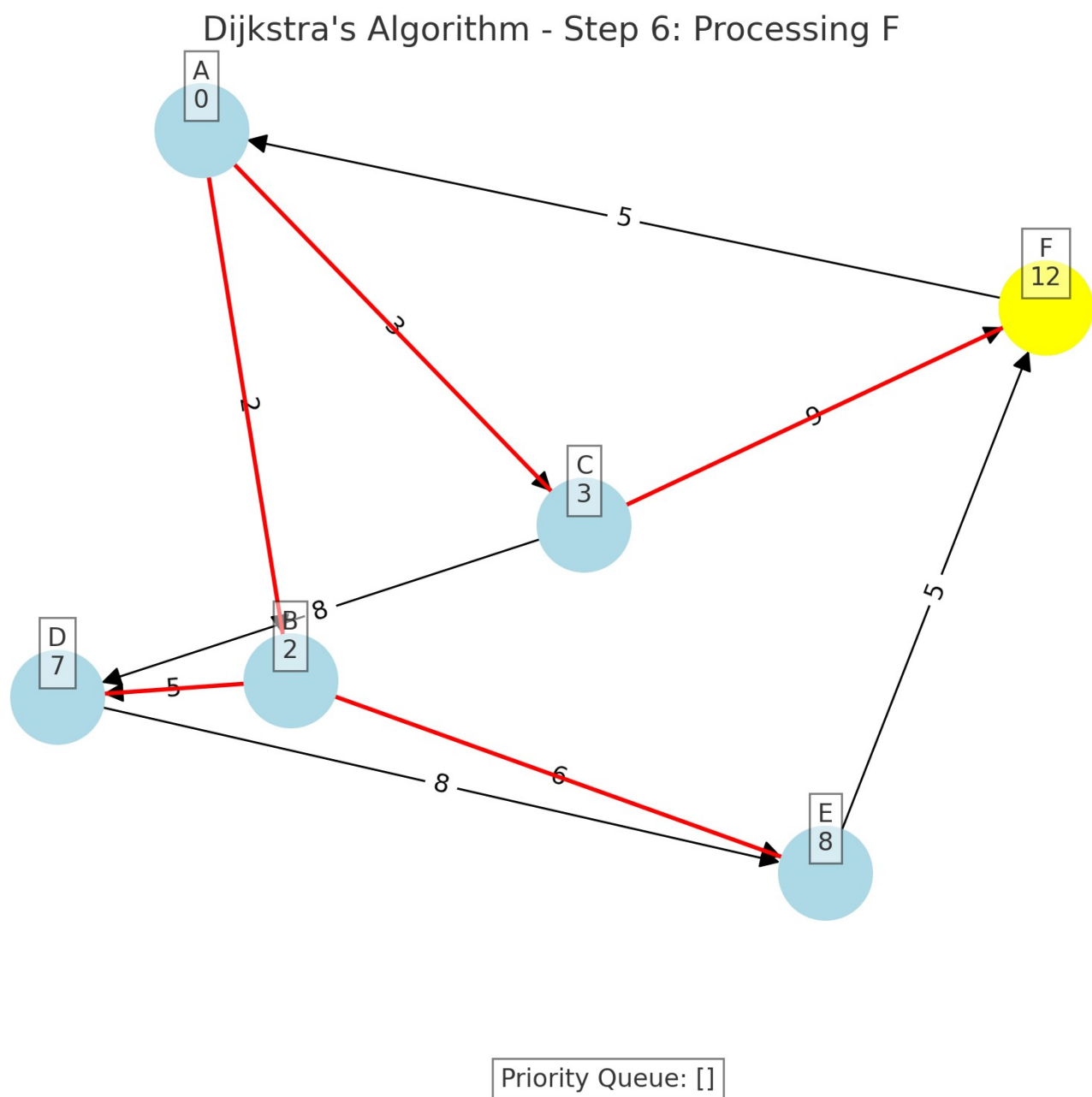
Dijkstra's Algorithm - Step 4: Processing D



Priority Queue: [('E', 8), ('F', 12)]

[6]





[6].

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

Adjacency List

This list is an adjacency list which is a list that allows nodes and edges connected to each other to show the relation. Through using this type of list, it can make every node a key or have some sort of element in the dictionary. I have attempted to make the graph for the question. It is here below.

graph = {

'A': [('B', 2), ('C', 3)], # A connects to B (weight 2) and C (weight 3)

'B': [('D', 5), ('E', 6)], # B connects to D (weight 5) and E (weight 6)

'C': [('D', 8), ('F', 9)], # C connects to D (weight 8) and F (weight 9)

'D': [('E', 8)], # D connects to E (weight 8)

'E': [('F', 5)], # E connects to F (weight 5)

'F': [('A', 5)] # F connects to A (weight 5)

}

Edge	Weight that occurred from student ID
A to B	2 (D1)
A to C	3 (D2)
B to D	5 (D3)
C to D	8 (D4)
B to E	6 (D5)

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

D to E	8 (D6)
C to F	9 (D7)
E to F	5 (D8)
F to A	5 (D9 = D1)

[7]

Edge Weight Table

[7]

Step by step for Dijkstra's Algorithm in table format

Iteration	Current Node	Priority Queue	Neighbours Processed	Distance Updates
1	A	[(0, A)]	B (2), C (3)	B: 2, C: 3
2	B	[(2, B), (3, C)]	D (5), E (6)	D: 7, E: 8
3	C	[(3, C), (7, D), (8, E)]	D (8), F (9)	No update for D, F: twelve
4	D	[(7, D), (8, E), (12, F)]	E (8)	No update for E
5	E	[(8, E), (12, F)]	F (5)	F: 13
6	F	[(12, F)]	A (2)	No update for A

[7]

Dijkstra's Algorithm Results: Shortest Paths and Distances

Destination	Shortest Path	Distance
A	A	0
B	A > B	2
C	A > C	3

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

D	$A > B < D$	7
E	$A > B > E$	8
F	$A > C > F$	13

[8]

Task 3:

(a) Create a Hash Table

Index	Letter	Occurrences
1	a	a-> a
2	b	b
3	c	
4	d	d
5	e	e
6	f	
7	g	
8	h	h -> h
9	i	i -> i
10	j	
11	k	
12	l	
13	m	m -> m -> m
14	n	
15	o	o
16	p	

Name: Mohamed Ibrahim

Assignment Title:

Algorithms Report – Coursework 2

Date of last update: 30/12/2024

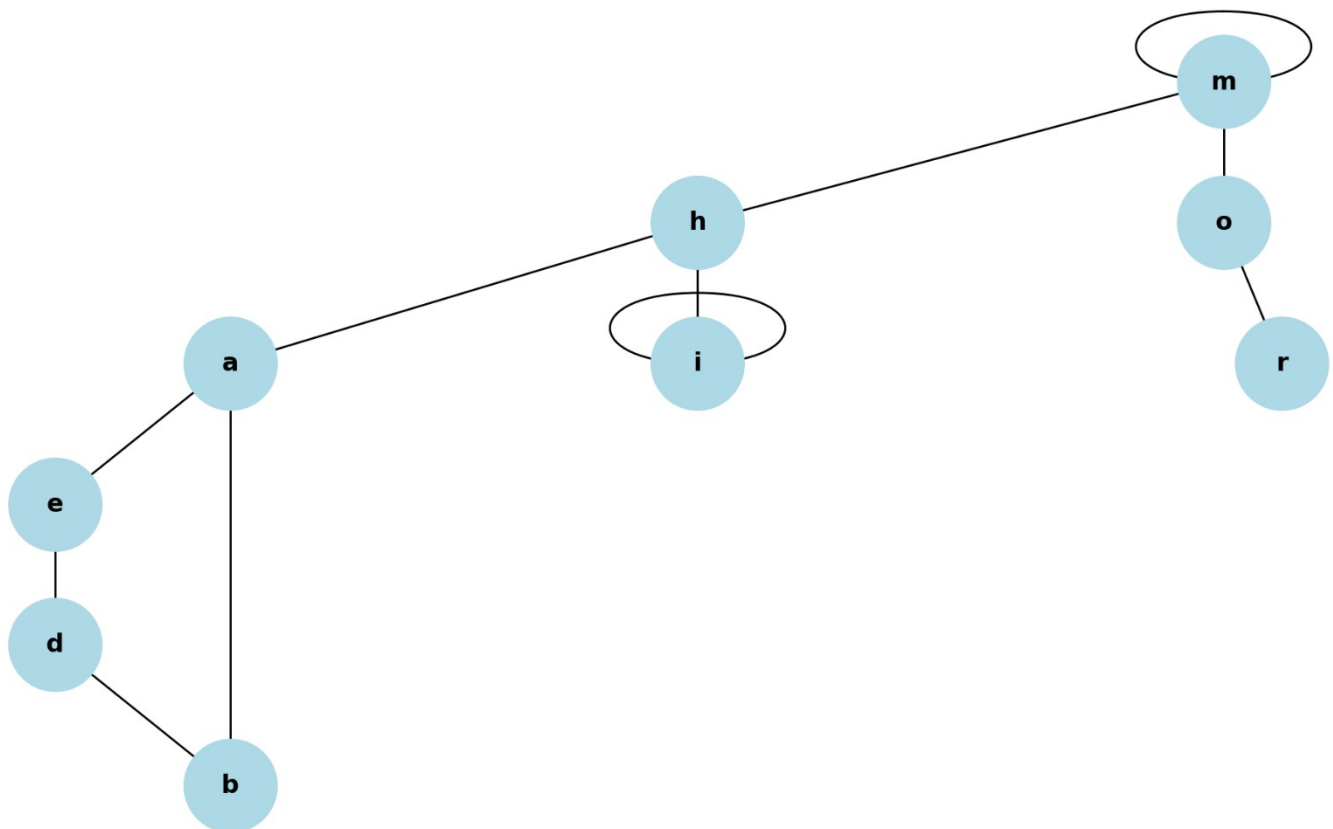
17	q	
18	r	r
19	s	
20	t	
21	u	
22	v	
23	w	
24	x	
25	y	
26	z	

[9]

[10]

(b) Create a Binary Search Tree (BST)

Binary Search Tree for 'Mohamed Ibrahim' (Including All Letters)



The Binary Search Tree (BST) represents the letters of my full name, "Mohamed Ibrahim," organized hierarchically. The construction process starts with the first letter (m) as the root node. Each subsequent letter is inserted into the tree based on its alphabetical comparison with the current nodes:

- If the letter is smaller, it is placed in the left subtree.
- If the letter is greater or equal, it is placed in the right subtree.

For example:

- The letter o, being greater than m, becomes the right child of m.
- The letter h, being less than m, becomes the left child of m [11].
- The letter a, being less than h, becomes the left child of h.

This process repeats until all letters are placed in the tree, including duplicates. The final BST structure is visualized as shown in the diagram, enabling efficient organization and search of the letters [11].

Conclusion

Three major tasks, such as implementation of Dijkstra's algorithm in solving the shortest-path problems in weighted graphs and providing detailed insights into the step-by-step workings and application and building a hash table that could map and retrieve data efficiently, were carried out with collision management using chaining. Additionally, the binary search tree demonstrated the hierarchical way of organizing data, which is highly useful in searching and sorting.

These exercises, therefore, outline the importance of algorithms and data

Name: Mohamed Ibrahim
Algorithms Report – Coursework 2

Assignment Title:

Date of last update: 30/12/2024

structures in solving real-world computational problems. These concepts form the basis on which complicated problems involving computer science, engineering, and data management are solved. Generally, this work demonstrates not only technical skill but also awareness of the overall importance of these techniques in modern technology.

Reference List

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT Press, 2001. Available: https://books.google.co.uk/books?id=NLngYyWFI_YC

[2]. GeeksforGeeks, "Merge Sort - GeeksforGeeks," *GeeksforGeeks*, Oct. 31, 2018. <https://www.geeksforgeeks.org/merge-sort/>

[3]. "Chapter 7: Sorting Algorithms Insertion Sort Mark Allen Weiss: Data Structures and Algorithm Analysis in Java Lydia Sinapova, Simpson College," *Slideplayer.com*, 2018. <https://slideplayer.com/slide/10148586/>

(accessed Dec. 17, 2024).

[4]. GeeksforGeeks, "Quick Sort Algorithm," *GeeksforGeeks*, Jan. 07, 2014. <https://www.geeksforgeeks.org/quick-sort-algorithm/>

Name: Mohamed Ibrahim
Algorithms Report – Coursework 2

Assignment Title:

Date of last update: 30/12/2024

[5]. Programiz, "Insertion Sort Algorithm," *Programiz.com*, 2020.

<https://www.programiz.com/dsa/insertion-sort>

[6]. GeeksforGeeks, "Dijkstra's algorithm," *GeeksforGeeks*, Nov. 19, 2018.

<https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>

[7]. "Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne," *algs4.cs.princeton.edu*.

<https://algs4.cs.princeton.edu/home/>

[8]. R. Python, "The Python heapq Module: Using Heaps and Priority Queues – Real Python,"

realpython.com. <https://realpython.com/python-heapq-module/>

[9]. GeeksforGeeks, "Hash Table Data Structure," *GeeksforGeeks*, Apr. 24, 2023.

<https://www.geeksforgeeks.org/hash-table-data-structure/>

[10]. programiz, "Hash Table," *Programiz.com*, 2019. [https://www.programiz.com/dsa/hash-](https://www.programiz.com/dsa/hash-table)

[table](https://www.programiz.com/dsa/hash-table)

[11]. Geeksforgeeks, "Binary Search Tree - GeeksforGeeks," *GeeksforGeeks*, 2015.

<https://www.geeksforgeeks.org/binary-search-tree-data-structure/>